

Fuente Tema 15: Entornos de desarrollo

```
string memory = ag;
```

```
try {
```

```
    conatormaia = string";
```

```
    if (annsightTimes) {
```

```
        const main = "";
```

```
        for (last g=1n <= 1++) {
```

```
            .string =
```

```
            .amocper";
```

```
            u, cu a ne
```

```
            marite = 3;
```

```
            if (wst += 40) {
```

```
                return ("head", "'# + "');
```

```
            }
```

```
        }
```

Visión de Rayos X

Dominando el arte de la depuración de código y el control de ejecución.

```
public enticoxtron(string args, etc) {
```

```
    longthe type = lning();
```

```
    .contoos.out.println(")+";
```

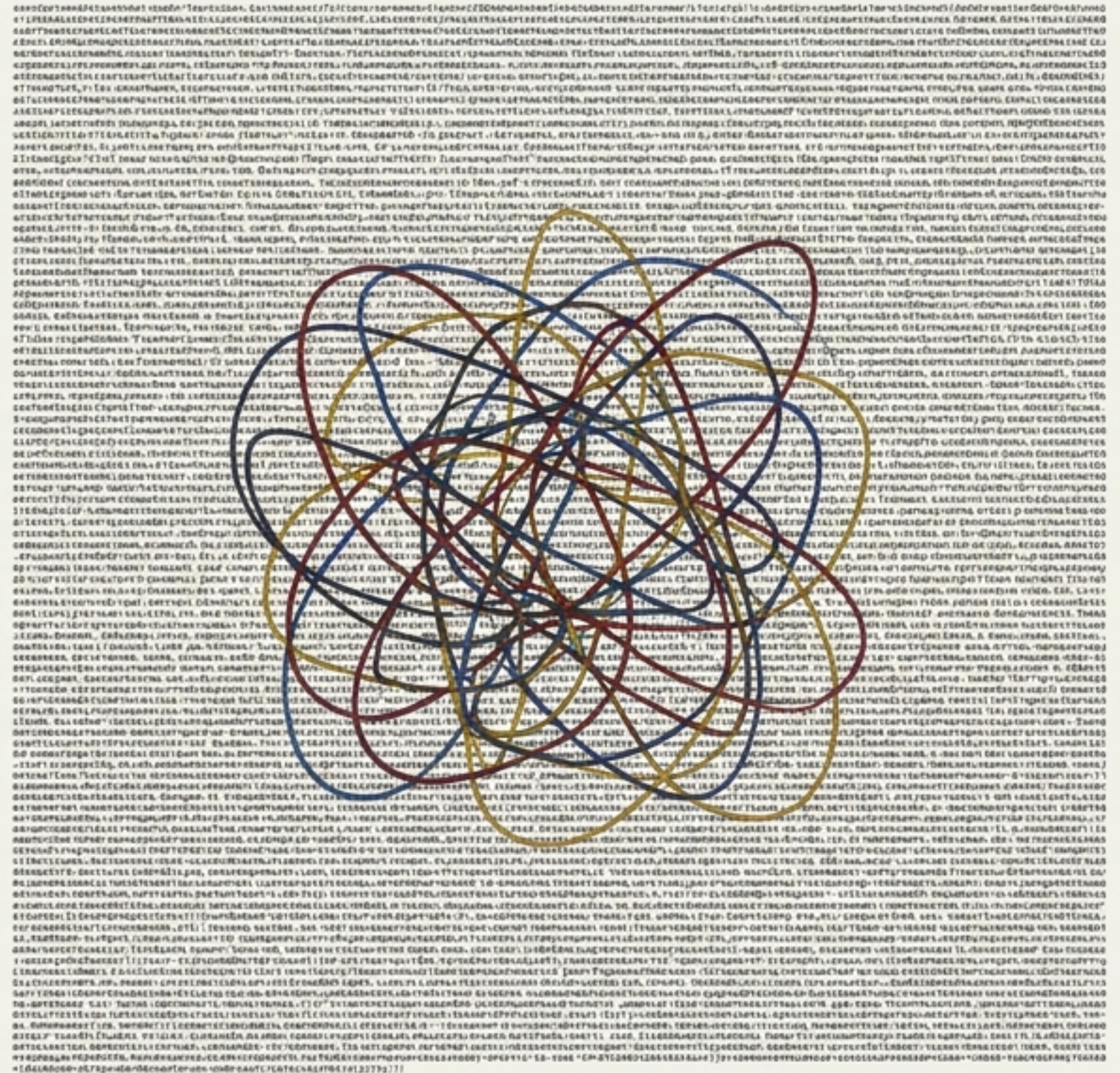
```
    return easto;
```


El problema de la opacidad a escala

A medida que aumentan las líneas de código, la complejidad del software crece exponencialmente.

Proyecto de +120.000 líneas de código. Soluciones concurrentes con múltiples hilos en ejecución.

En este nivel de escala, los errores lógicos en tiempo de ejecución se vuelven invisibles. Depender únicamente de la experiencia del desarrollador es volar a ciegas.



El Depurador: Inyectando transparencia

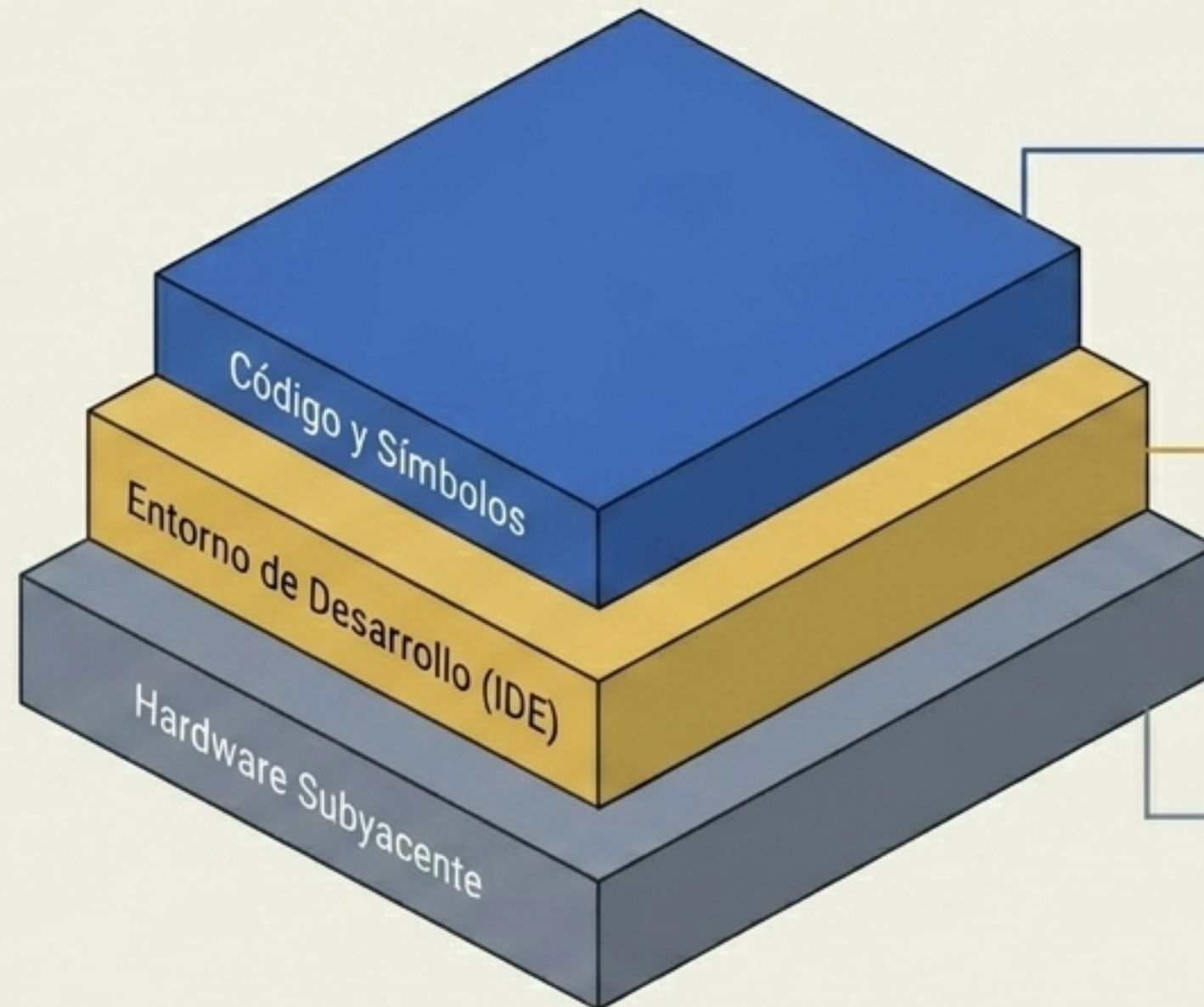
[01] **Símbolos Especiales:** La versión de depuración no es idéntica a la de producción. Inyecta símbolos especiales para controlar el flujo.

[02] **Sacrificio de Rendimiento:** La ejecución es ligeramente más lenta debido a los símbolos, pero ofrece control absoluto.

[03] **Control de Flujo:** Permite examinar qué instrucciones exactas se ejecutan en cada milisegundo.

```
function caaolot(RestonooReesendena.stone) {  
  function lintsehair() {  
    if (ondueckant == null) {  
      // soscesusaenz te tide so conoplete  
      return null;  
    }  
  
    return null;  
  }  
  
  function isreanExtent(terotciser) {  
    variabls = o:oatt;  
    conet wotil = o:ses();  
    if (sla.aegsst) {  
      return applicabloung: "akb/7a50"xtrüüqf'drueritae3";  
    }  
  }  
  
  // Macrate ins  
  bunns:={  
    if (vitroteo  
    nesolub  
    return o  
  } else if m  
    return mu  
  }  
  
  // Deutráto  
  if (arocosen  
    return, so  
  }  
  
  return ttdi  
}  
  
function precasaStrot() {  
  ver octolts = stoll9;  
  var cabos = detl.ctateSttoki();  
  var anotetson = amidetota;  
  // portaalérdees +3: 548y  
  var seasochn = soncion steesao.feonltg,BottodDiscalopSeotar, r8ppfx"ceorade  
  // oanteced r= 100  
  gae.cnoetacrcócttaritfrenopesl3ilrll);  
  var.cóhcf.eondop = artaduesidactodll);  
  if (oala == 11 {  
    // 100.000 s/narrátes va drcplongfame  
    car-stezoare-entoro.contategolionleocerélsnudefaut();  
    if (laa0tgs == o:1) {  
      soecceolols.epothoDebug("Vextk", "Úejokceof: su-s10ms vwt.breonepl3  
    } else {  
      sorenas369las:snetchyusslnkfynolotion vo-arbow-se-deuica, carest2el;  
    }  
  }  
  
  var dvolitot-ar e-s-logs.contatego.tabletatwae,defaut();  
  if lóreannecho01028 == t711) {  
    toturn.oott:totoc0478vesotsonPanglto:otaten('heepuctr "Vasab 4 "W"o
```


Anatomía de la interrupción: La pila de dependencias



El software debe estar **compilado** con información y **símbolos** de depuración.

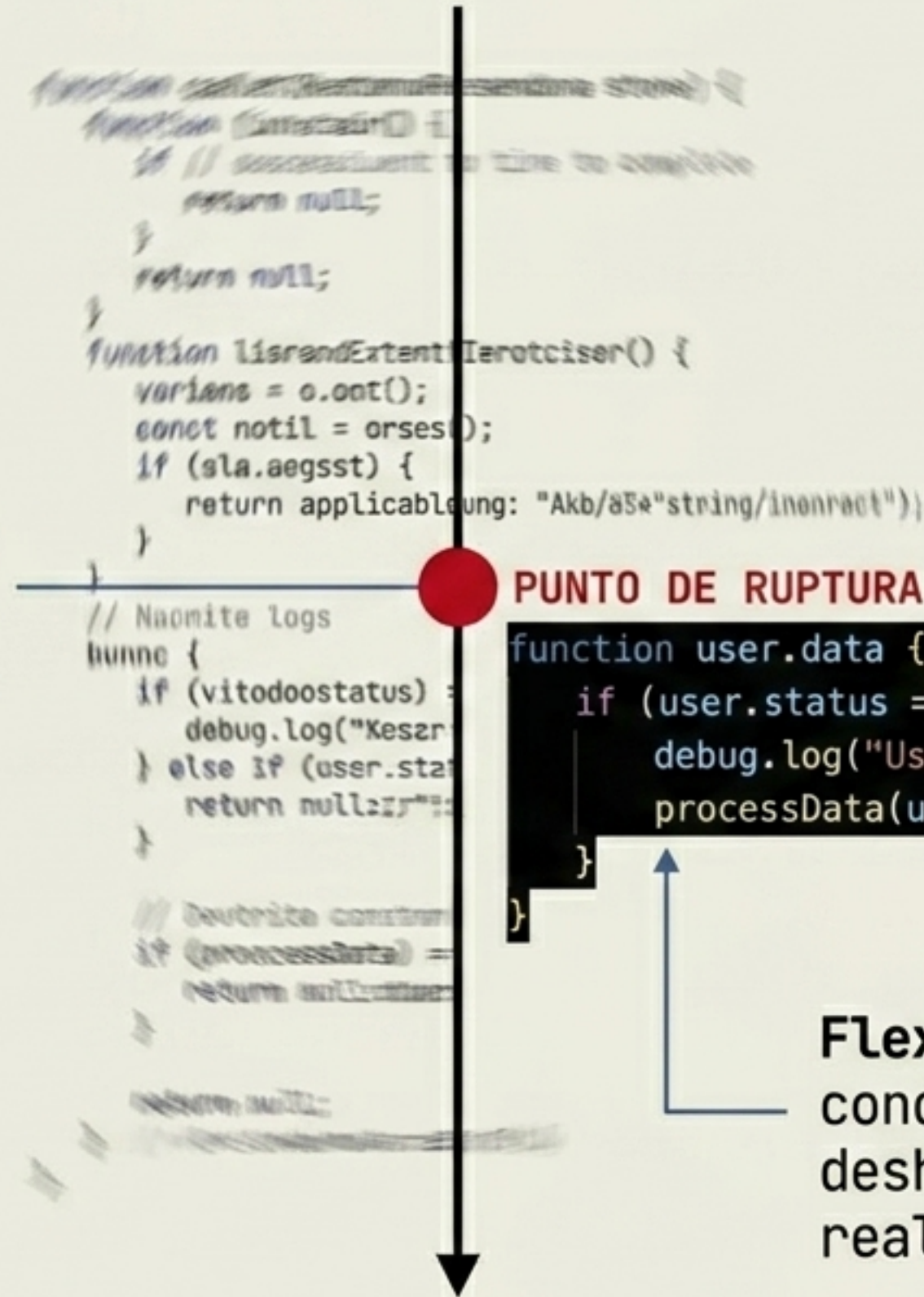
El software del entorno debe soportar la **interpretación** y **visualización** de la depuración.

La **arquitectura física** (ej. registros x86) debe proveer soporte nativo para **interrumpir** el flujo de ejecución y proteger la memoria.

La depuración no es solo una función del programa; requiere soporte sincronizado desde el silicio hasta la interfaz.

Detener el tiempo: El Punto de Ruptura

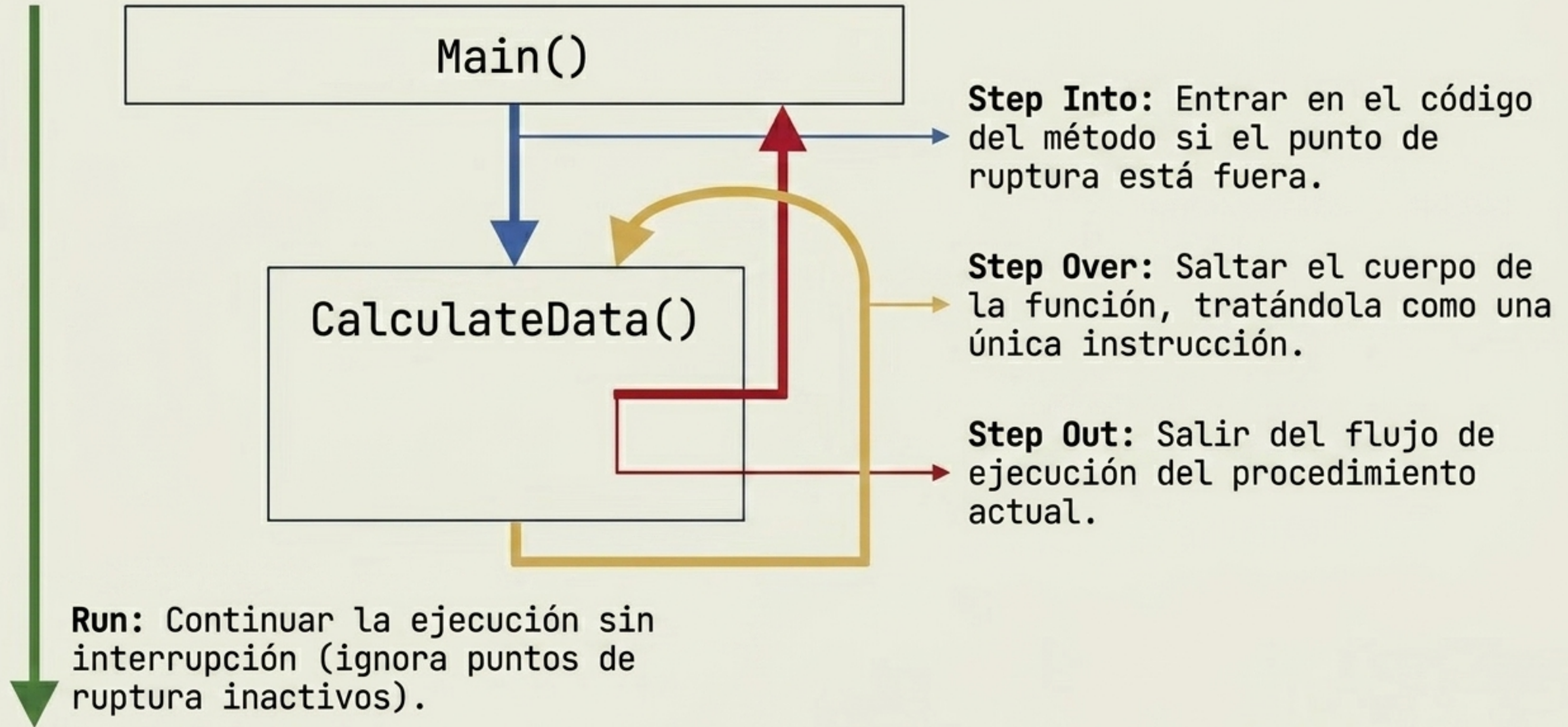
Pausa Absoluta: El software detiene su ejecución por completo sin realizar ninguna tarea adicional.



Visibilidad Total: El programador asume el control para inspeccionar variables, logs y estado de la memoria.

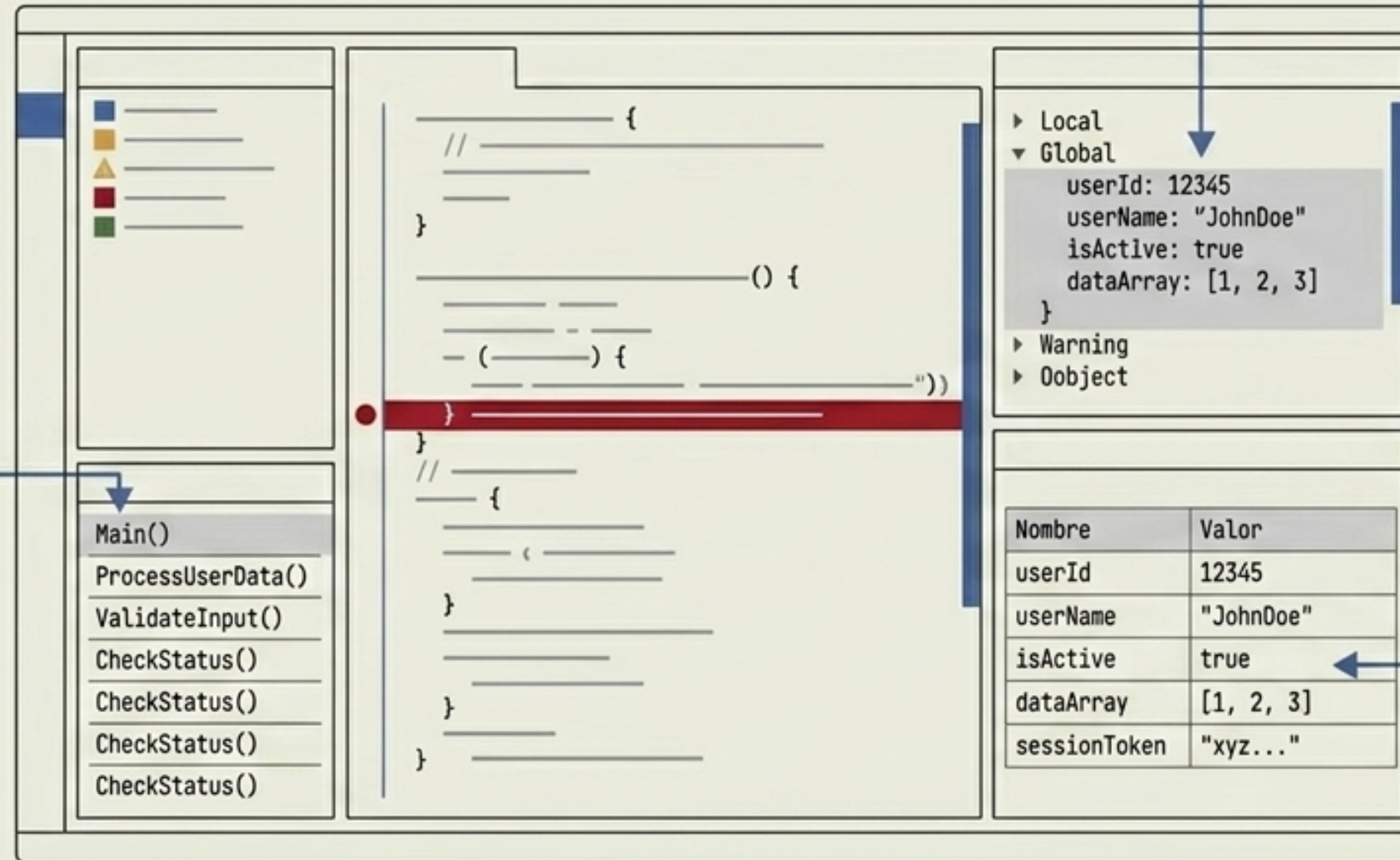
Flexibilidad: Se pueden ubicar en condicionales (if), habilitar, deshabilitar o modificar en tiempo real.

Navegando el tiempo: Seguimiento en ejecución



El Panel de Diagnóstico: Examinadores de Estado

Pila de Ejecución (Call Stack): Rastrea el historial de llamadas a funciones que nos trajo hasta este punto exacto.



Ámbito de Visibilidad (Scope): Aísla y muestra únicamente la porción de código y memoria que está activa en este momento.

Examinador de Variables: Revela el contenido exacto de los datos almacenados, crucial en lenguajes de tipado no intuitivo.

Diagnóstico de Estrategias: Logs vs. Depurador

	Desarrollo Basado en Logs	Desarrollo con Depurador
Naturaleza del Análisis	Análisis <i>post-mortem</i> (leer el pasado).	Intervención en <i>tiempo real</i> (congelar el presente).
Granularidad	<i>Limitada</i> a donde el programador recordó poner un <i>log</i> .	<i>Absoluta</i> . Acceso al estado completo de la memoria y la pila.
Viabilidad a Gran Escala	<i>Ineficiente</i> . Intentar depurar código ajeno mediante <i>logs</i> es <i>tedioso</i> .	<i>Indispensable</i> . Permite aislar funciones específicas <i>paso a paso</i> .
Rendimiento	<i>Rápido en producción</i> , <i>lento</i> para diagnosticar.	Requiere binarios no optimizados y consume <i>más memoria</i> , pero reduce drásticamente el tiempo de resolución.

El veredicto sobre el entorno

“A medida que el número de líneas de código aumenta, la ausencia de un depurador equivale a operar a ciegas.”



Elección Estratégica:

La elección del IDE y el lenguaje debe garantizar herramientas de depuración nativas desde el día cero.



Optimización del Tiempo:

Sustituye la dependencia en la intuición del programador por diagnósticos precisos del flujo de ejecución.



Soporte Integral:

Requiere que la infraestructura –desde la compilación sin optimizar hasta la arquitectura hardware– trabaje en unísono.